

Ricorsione

Table of contents

La Ricorsione	1
Esempio: il problema degli 11 cammelli.	1
Osservazione:	2
Definizione	3
Tipi di ricorsione (casi principali)	4
Ricorsione lineare (diretta)	4
Come funziona lo stack delle chiamate	5
Ricorsione multipla	5
Fibonacci iterativo in Python	6
Ricorsione mutua o indiretta	7
Ricorsione vs iterazione	8
Limiti pratici dello stack	8
Next step	9

La Ricorsione

Esempio: il problema degli 11 cammelli.

Un mercante di cammelli lascia ai suoi tre figli i suoi 11 cammelli, stabilendo che la ripartizione dell'asse ereditario sarà la seguente

- primo figlio $\rightarrow 1/2$
- secondo figlio $\rightarrow 1/4$
- terzo figlio $\rightarrow 1/6$

Si vede che resta $1/12$ indiviso. Il testamento si può interpretare nel senso che la quota indivisa va perduta (per esempio in beneficenza), oppure che la quota indivisa va ripartita secondo le percentuali dette.

Seguendo la seconda interpretazione, qual è l'eredità di ciascun figlio?

Sia:

$$\text{rimanenza} = \frac{1}{12}$$

A ogni iterazione, la parte non assegnata viene nuovamente suddivisa con le stesse porzioni.

Tabella delle quote assegnate a ogni iterazione:

	Iterazione 1	Iterazione 2	Iterazione 3	Iterazione 4
Figlio 1 ($\frac{1}{2}$)	$11 \cdot \frac{1}{2}$	$11 \cdot \frac{1}{2} + \frac{11}{12} \cdot \frac{1}{2}$	$\frac{11}{288}$	$\frac{11}{3456}$
Figlio 2 ($\frac{1}{4}$)	$11 \cdot \frac{1}{4}$	$11 \cdot \frac{1}{4} + \frac{11}{12} \cdot \frac{1}{4}$	$\frac{11}{576}$	$\frac{11}{6912}$
Figlio 3 ($\frac{1}{6}$)	$11 \cdot \frac{1}{6}$	$11 \cdot \frac{1}{6} + \frac{11}{12} \cdot \frac{1}{6}$	$\frac{11}{864}$	$\frac{11}{10368}$
Rimanenza ($\frac{1}{12}$)	$\frac{11}{12}$	$\frac{11}{12^2}$	$\frac{11}{12^3}$	$\frac{11}{12^4}$

Osservazione:

A ogni iterazione la rimanenza viene moltiplicata per $\frac{1}{12}$, quindi le quote formano serie geometriche.

Il primo figlio riceve:

.

$$\frac{11}{2} + \frac{11}{24} + \frac{11}{288} + \dots$$

cioè

$$\frac{11}{2} \left(1 + \frac{1}{12} + \frac{1}{12^2} + \dots \right)$$

La serie geometrica vale:

$$\sum_{k=0}^{\infty} \left(\frac{1}{12} \right)^k = \frac{1}{1 - \frac{1}{12}} = \frac{12}{11}$$

Quindi:

- Figlio 1: $\frac{11}{2} \cdot \frac{12}{11} = 6$
- Figlio 2: $\frac{11}{4} \cdot \frac{12}{11} = 3$
- Figlio 3: $\frac{11}{6} \cdot \frac{12}{11} = 2$

La funzione chiama ricorsivamente se stessa finché la quota indivisa è trascurabile.

```
def eredita(asse, pf1, pf2, pf3, soglia=0.0001):
    # caso base: la rimanenza è trascurabile
    if asse < soglia:
        return (0, 0, 0)

    # quote correnti
    v_f1 = asse * pf1
    v_f2 = asse * pf2
    v_f3 = asse * pf3

    # parte non assegnata
    rimanenza = asse - (v_f1 + v_f2 + v_f3)

    # distribuzione ricorsiva della rimanenza
    r1, r2, r3 = eredita(rimanenza, pf1, pf2, pf3, soglia)

    # totale = quota corrente + quota ricorsiva
    return (
        v_f1 + r1,
        v_f2 + r2,
        v_f3 + r3
    )
```

Definizione

Una funzione è **ricorsiva** se richiama se stessa direttamente o indirettamente.

Ogni definizione ricorsiva richiede:

1. **Caso base** — condizione che termina la ricorsione (non genera nuove chiamate).
2. **Caso ricorsivo** — il problema viene ridotto a una versione più piccola dello stesso problema. .

Senza caso base la ricorsione non termina portando ad una saturazione della memoria (*stack overflow*).

Tipi di ricorsione (casi principali)

Ricorsione lineare (diretta)

La funzione chiama se stessa **una sola volta** per esecuzione.

Esempio: fattoriale

$$n! = \begin{cases} n \cdot (n-1)! & \text{se } n > 1 \\ 1 & \text{se } n = 0 \text{ o } n = 1 \end{cases}$$

Fattoriale — C

```
int fattoriale(int n) {  
    if (n <= 1) return 1;           // caso base  
    return n * fattoriale(n - 1);  // caso ricorsivo  
}
```

Fattoriale — Python

```
# NB: annotazioni di tipo opzionali  
def fattoriale(n: int) -> int:  
    if n <= 1:                # caso base  
        return 1  
    return n * fattoriale(n - 1)
```

ESECUZIONE

```
fattoriale(5)  
= 5 * fattoriale(4)  
= 5 * 4 * fattoriale(3)  
= 5 * 4 * 3 * fattoriale(2)  
= 5 * 4 * 3 * 2 * fattoriale(1)  
= 5 * 4 * 3 * 2 * 1 * fattoriale(0)  
= 5 * 4 * 3 * 2 * 1 * 1 =  
= 120
```

Come funziona lo stack delle chiamate

Ogni chiamata a funzione crea un nuovo contesto di esecuzione.

Le chiamate ricorsive vengono memorizzate nello **stack delle chiamate**.

Con la ricorsione i frame si accumulano fino al caso base, poi vengono rimossi (unwinding) restituendo i valori verso il chiamante originale.

```
fattoriale(3)
  fattoriale(2)
    fattoriale(1)
      fattoriale(0)  ← caso base: rende 1
    ← rende 1*1 = 1
  ← rende 2*1 = 2
← rende 3*2 = 6
```

Ricorsione multipla

La funzione chiama se stessa **più di una volta** per esecuzione. L'albero delle chiamate cresce esponenzialmente.

Esempio: sequenza di Fibonacci https://it.wikipedia.org/wiki/Successione_di_Fibonacci

$$F_n = \begin{cases} 0 & \text{se } n = 0 \\ 1 & \text{se } n = 1 \\ F_{n-1} + F_{n-2} & \text{se } n > 1 \end{cases}$$

Sequenza:

[0, 1, 1, 2, 3, 5, 8, 13, 21, ...]

Questo è un esempio di algoritmo **non adatto a implementazione ricorsiva** perché ripete molte chiamate inutili.

Fibonacci — C

```
int fibonacci(int n) {
    if (n <= 1) return n;
    return fibonacci(n - 1) + fibonacci(n - 2);
}
```

Fibonacci — Python

```
def fibonacci(n: int) -> int:
    if n <= 1:
        return n
    return fibonacci(n - 1) + fibonacci(n - 2)
```

Chiamate:

```
fibonacci(5)
  fibonacci(4)
    fibonacci(3)
      fibonacci(2)
        fibonacci(3)
```

Come si vede, `fibonacci(3)` viene calcolato più volte. In questi casi una soluzione iterativa è spesso migliore.

Attenzione: `fibonacci(40)` genera ~2,5 miliardi di chiamate, mentre l'implementazione iterativa ha un costo di $O(n)$. Questo è un caso in cui la ricorsione **non** è la scelta ottima.

Fibonacci iterativo in Python

L'idea dell'algoritmo iterativo è mantenere sempre in memoria solo gli ultimi due numeri della sequenza.

```
def fibonacci_iterativo(n):
    previous = 0
    current = 1

    for i in range(n):
        next_value = previous + current

        previous = current
        current = next_value

    return previous
```

n	F_n (previous)	current	next_value
0	0	1	1
1	1	1	2
2	1	2	3
3	2	3	5

n	F_n (previous)	current	next_value
4	3	5	8
5	5	8	13
6	8	13	21
7	13	21	...

Ricorsione mutua o indiretta

Due o più funzioni si chiamano a vicenda.

$$f(n) = k_1 \cdot g(n-1)$$

$$g(n) = k_2 \cdot f(n-1)$$

Esempio

C

```
int is_even(int n);
int is_odd(int n);

int is_even(int n) {
    if (n == 0) return 1;
    return is_odd(n - 1);
}

int is_odd(int n) {
    if (n == 0) return 0;
    return is_even(n - 1);
}
```

Python

```
def is_even(n: int) -> bool:
    if n == 0: return True
    return is_odd(n - 1)

def is_odd(n: int) -> bool:
    if n == 0: return False
    return is_even(n - 1)
```

Esempi meno banali di ricorsione indiretta sono i parser o l'attraversamento dei grafi. ***

Ricorsione vs iterazione

Criterio	Ricorsione	Iterazione
Leggibilità	Alta su problemi naturalmente ricorsivi	Alta su problemi lineari
Stack	Usa lo stack di sistema (limite!)	Nessun overhead di stack
Efficienza	Overhead per ogni chiamata	In genere più veloce
Stato	Implicito nei frame	Esplicito nelle variabili

Regola pratica:

- Usare la ricorsione è comodo quando il problema ha struttura ricorsiva naturale (alberi, suddivisioni del dominio, backtracking), e quindi la formulazione dell'algoritmo è immediata con la ricorsione. Tuttavia spesso, per questioni di ottimizzazione si trasforma l'algoritmo ricorsivo in un algoritmo iterativo,
- In alcuni casi è meglio usare direttamente un approccio iterativo (somme, ricerche sequenziali).

Limiti pratici dello stack

In C lo stack di default è tipicamente 1–8 MB.

In Python il limite di default è **1000 frame** (`sys.getrecursionlimit()`).

```
import sys
print(sys.getrecursionlimit()) # 1000
sys.setrecursionlimit(5000)    # aumentabile, ma con cautela
```

Per ricorsioni profonde su strutture dati (liste concatenate, alberi sbilanciati) occorre valutare alternative iterative o aumentare il limite consapevolmente.

Next step

La ricorsione è la base per comprendere:

- **Algoritmi divide et impera** (Merge Sort, Quick Sort)
 - **Deallocazione ricorsiva** di strutture dati dinamiche (linked list, alberi)
 - **Backtracking** (esplorazione di spazi di ricerca)
 - **Analisi della complessità computazionale** ricorsiva (relazioni di ricorrenza)
-

Esercizi

Implementare in modo ricorsivo e non ricorsivo

- Fattoriale
- https://en.wikipedia.org/wiki/Triangular_number
- Fibonacci
- Implementare in C la soluzione dell'esercizio dei cammelli
- Deallocare uno stack implementato mediante lista concatenata

Traccia:

```
void free_stack(Node* top) {
    if (top == NULL) {
        return;
    }

    free_stack(top->next);
    free(top);
}
```

- Deallocare una coda implementato mediante lista concatenata

```
void free_queue_nodes(Node* node) {
    if (node == NULL) {
        return;
    }

    free_queue_nodes(node->next);
    free(node);
}

void free_queue(Queue* queue) {
    free_queue_nodes(queue->front);
}
```

```
    free(queue);  
}
```